CLOUD COMPUTING — PROJECT PRESENTATION

Cloud Computing on Amazon Web Services

Market position, service architecture, virtual machine provisioning and cost engineering — with a reference implementation that runs.

Student ..... · ID .....

Course ..... · Instructor .....

Date .....

Every figure sourced in data/figures.json · prices retrieved 7 Sept 2026

AGENDA

What this presentation covers

1. **What cloud computing is** — and where the responsibility boundary sits
2. **The market** — who leads, who is gaining, and how fast it is growing
3. **AWS infrastructure** — regions, availability zones, VPCs
4. **Anatomy of a virtual machine** — the eight decisions in one launch
5. **Setting one up** — console, CLI, and infrastructure as code
6. **Reference architecture** — the data flow that survives a failure
7. **Pricing** — where the money actually goes
8. **Use cases, security, and the comparison**

1 — DEFINITION

Cloud computing, per NIST

On-demand network access to a shared pool of configurable computing resources, rapidly provisioned and released with minimal management effort.

- **On-demand self-service** — no purchase order, no ticket, no human
- **Broad network access** — standard protocols, any client
- **Resource pooling** — your VM is a slice of a bigger machine
- **Rapid elasticity** — capacity scales out and back in
- **Measured service** — metered per instance-second, per GB-month, per request

> **All five, or it is not cloud.** Measured service is the one that turns an architecture decision into a line on an invoice — which is why half of this presentation is about cost.

IaaS, PaaS, SaaS: where your responsibility ends

One quarter of cloud spending

$143.4B
Worldwide cloud infrastructure spend, Q2 2026 — a single quarter

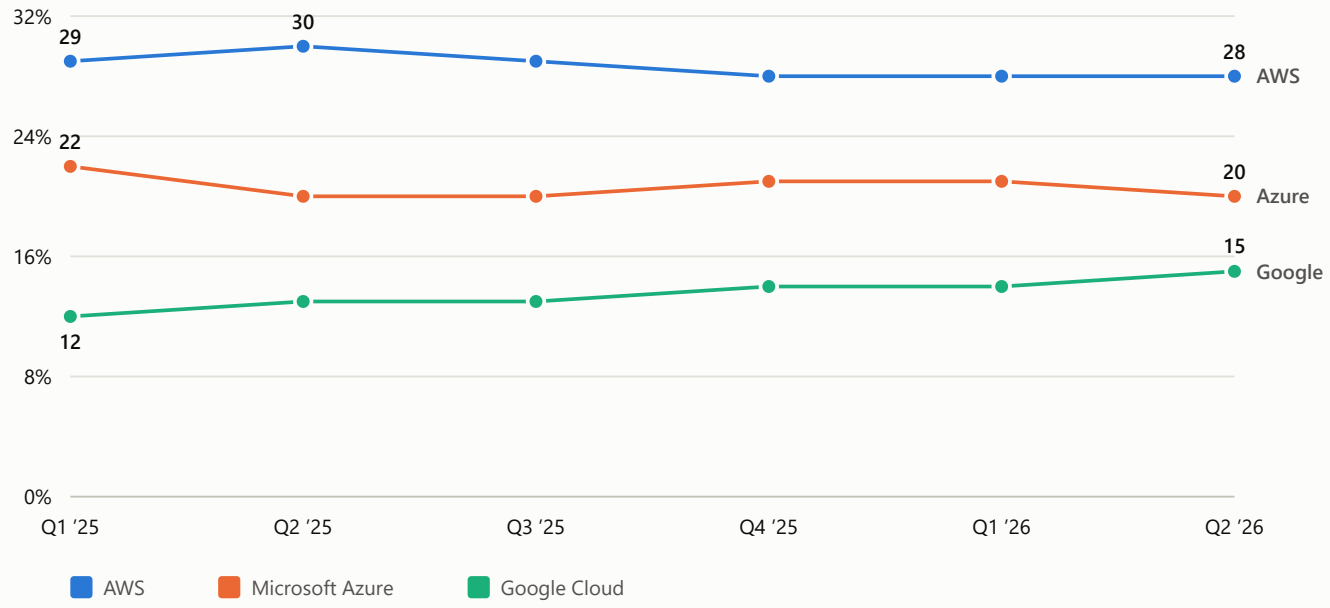
43%
Year-on-year growth — highest in eight years, 11th successive quarter of acceleration

$500B
Trailing twelve-month market revenue

67%
Of public cloud held by AWS, Azure and Google combined

The market has **doubled in eleven quarters**. Synergy attributes the acceleration primarily to generative AI: GenAI-specific cloud services grew **165% year on year** in the same quarter.

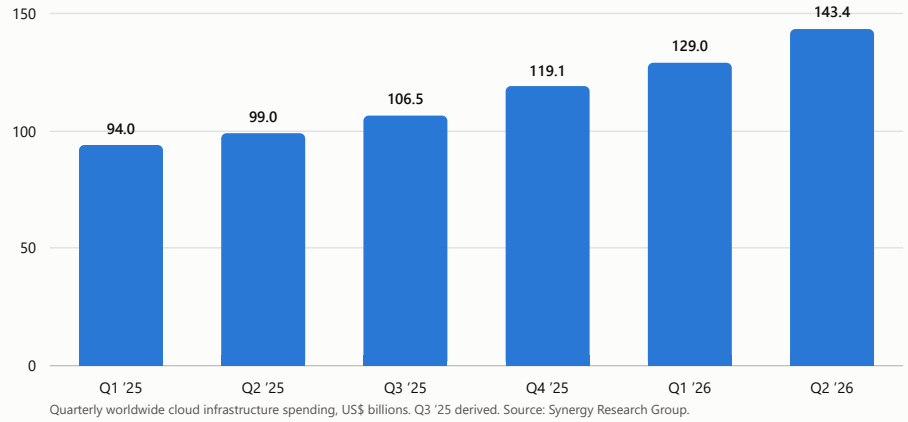
Market share: AWS leads, Google gains



Percent of worldwide enterprise spending on cloud infrastructure services. Source: Synergy Research Group quarterly releases.

Report Figure 2 · AWS −2 points, Google +3 points over six quarters

…but share of *what*?



Quarterly worldwide cloud infrastructure spending, US$ billions. Q3 '25 derived. Source: Synergy Research Group.

+52.6%

Market growth across these six quarters (143.4 ÷ 94.0).

AWS lost **two points of share** across a market that grew by **half**. Its absolute revenue rose substantially over the same period.

A falling share of a market growing 43% a year is not a decline.

2 — THE CHOICE

Why this project examines AWS

1 · It is the largest

28% of a $143bn quarter. More workloads run here than anywhere else, so understanding it is the most consequential.

2 · It is the oldest

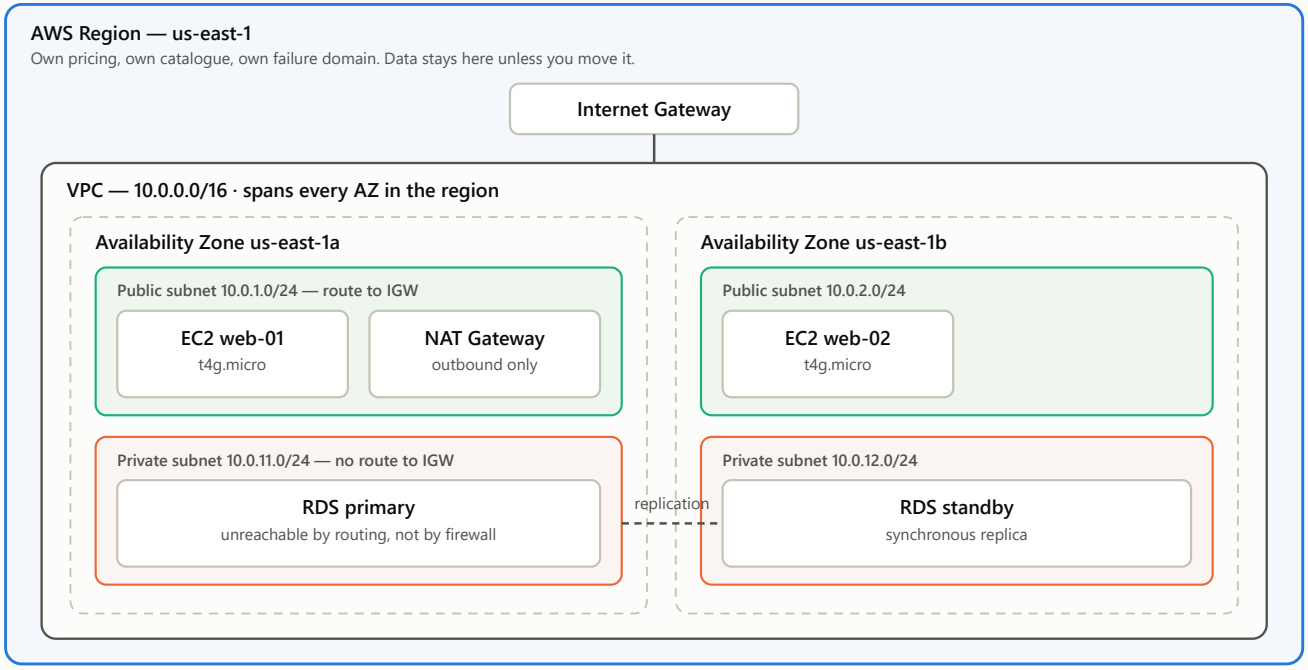
EC2 launched in 2006. Its abstractions — instance, AMI, security group, availability zone — were copied by its competitors, so the knowledge transfers almost directly to Azure and Google Cloud.

3 · Its pricing is public and machine-readable

Every price in this presentation was read from the same feed the AWS pricing page reads. The cost analysis can be *verified*, not just believed.

This is not an endorsement — slide 21 compares all three, and slide 22 sets out what choosing AWS costs you.

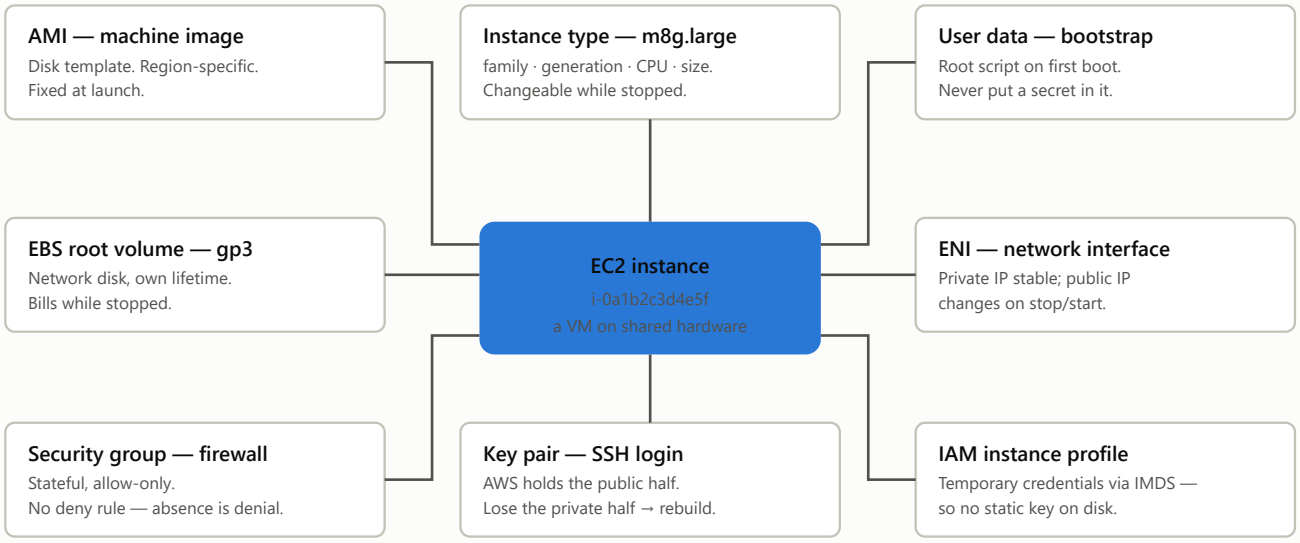
Block diagram: region → AZ → VPC → subnet → instance



A subnet lives in exactly one AZ; a VPC spans them all. That asymmetry is what makes multi-AZ design a subnet decision.

4 — ANATOMY

One EC2 instance = eight decisions



Fixed for the life of the instance: the AMI, the placement (subnet → AZ), and the key pair name. Everything else is changeable.

5 — PROVISIONING

The same VM, three ways

CRITERION	CONSOLE	AWS CLI	TERRAFORM
Time to first instance	~3 minutes	~90 seconds	~2 minutes
Repeatable identically	No	Yes — but duplicates on re-run	**Yes — re-run converges**
Reviewable before applying	No	No	**Yes —** `terraform plan`
Clean removal	Manual, easy to orphan	Scripted, tag-based	`terraform destroy`
Best used for	Learning; one-off inspection	Automation inside scripts, CI	Anything that must exist tomorrow

Held constant across all three: us-east-1 · Amazon Linux 2023 arm64 · `t4g.micro` · 20 GiB encrypted gp3 · SSH from one IP, HTTP from anywhere · IMDSv2 required · nginx via user data.

5.1 — CONSOLE

Ten clicks, and the four that matter

1. **Region first** — everything after is scoped to it
2. EC2 → Instances → **Launch an instance**
3. **Name and tags** — untagged estates become unbillable
4. **AMI** — Amazon Linux 2023, architecture *64-bit Arm*
5. **Instance type** — `t4g.micro`
6. **Key pair** — ed25519, downloaded *once*
7. **Network** — subnet, public IP on, security group
8. **Storage** — 20 GiB gp3, encrypted
9. **Advanced** — metadata *V2 only*, paste user data
10. **Launch** — wait for status checks *2/2*, not just *running*

Where marks are lost

SSH source. The console offers "Anywhere" in one click. Choose *My IP*.

The key file. Downloaded once. No recovery — lose it and you rebuild.

Metadata version. Leave it at V1-and-V2 and one SSRF bug reads your IAM credentials.

Architecture mismatch. An x86 AMI will not launch on a Graviton type.

The console is the right *first* path — it names every decision out loud. It is the wrong *tenth* path, because nothing you clicked is written down.

One command, and two habits worth more than it

```
aws ec2 run-instances --region us-east-1 \
  --image-id "$AMI_ID" --instance-type t4g.micro \
  --key-name cloud-project-key --security-group-ids "$SG_ID" \
  --user-data file://scripts/user-data.sh \
  --metadata-options "HttpTokens=required,HttpEndpoint=enabled,HttpPutResponseHopLimit=1" \
  --block-device-mappings '[{"DeviceName":"/dev/xvda","Ebs":{"VolumeSize":20,
       "VolumeType":"gp3","Encrypted":true,"DeleteOnTermination":true}}]'
```

Resolve the AMI, never paste it

```
aws ssm get-parameters --names \
 /aws/service/ami-amazon-linux-latest/\
al2023-ami-kernel-default-arm64
```

Compute your own IP, never type /0

```
MY_IP="$(curl -s https://checkip.amazonaws.com)/32"
--ip-permissions "...IpRanges=[{CidrIp=$MY_IP}]"
```

What this path exposes: run it twice and you get two instances, two security groups, two bills. It *creates* resources but does not *track* them — which is the entire argument for the next slide.

The instance, written down

```
resource "aws_instance" "vm" {
  ami                    = data.aws_ami.al2023_arm.id
  instance_type          = var.instance_type
  subnet_id              = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name               = aws_key_pair.this.key_name
  user_data              = file("../scripts/user-data.sh")

  root_block_device {
    volume_size = 20
    volume_type = "gp3"
    encrypted   = true
    delete_on_termination = true
  }
  metadata_options { http_tokens = "required" }
  tags = { Name = var.name }
}
```

plan before apply

You see what will change *before* anything changes.

diffable, reviewable

Infrastructure goes through code review like code.

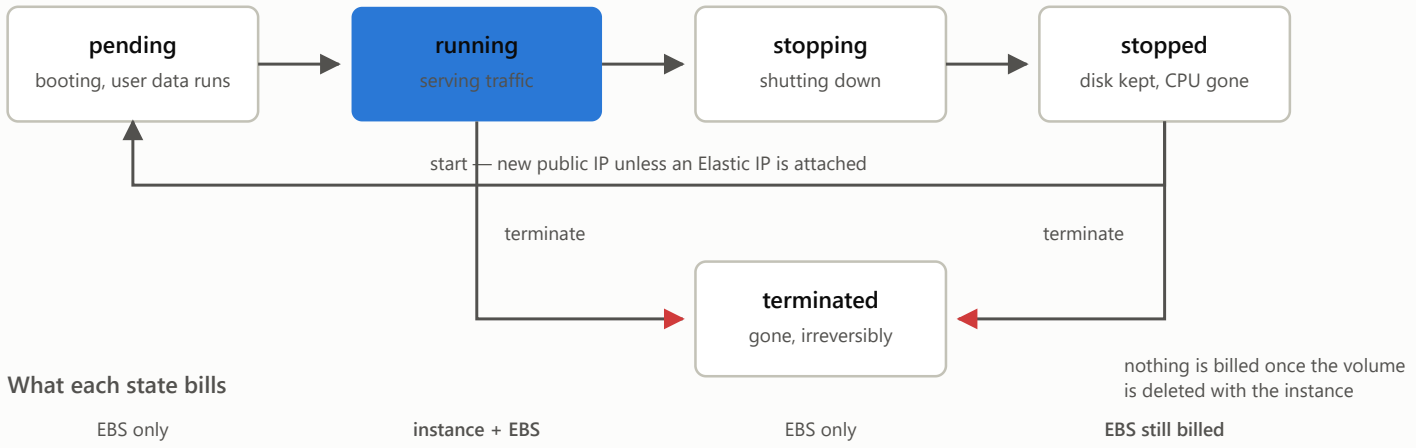
destroy is exact

Removes what it created — the difference between a $6 exercise and a $60 one.

`variables.tf` rejects `0.0.0.0/0` outright, so the most common security mistake in this assignment fails at *plan* time.

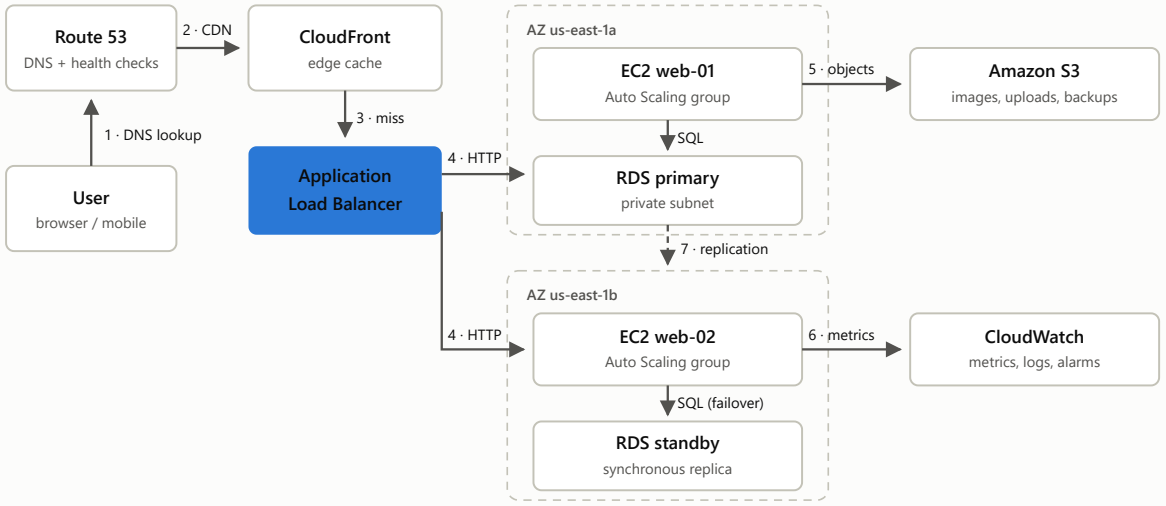
5.5 — LIFECYCLE

“Stopped” is not “free”



A stopped instance stops the instance-hour charge but its EBS volume keeps billing, **per GB-month, for as long as the volume exists**. Only **terminate** ends the charge. This one distinction accounts for most surprise bills in student accounts.

Data flow: the same app, made survivable



3 · The cheapest request is the one your instances never see. 4 · The load balancer removes an unhealthy instance without human action.

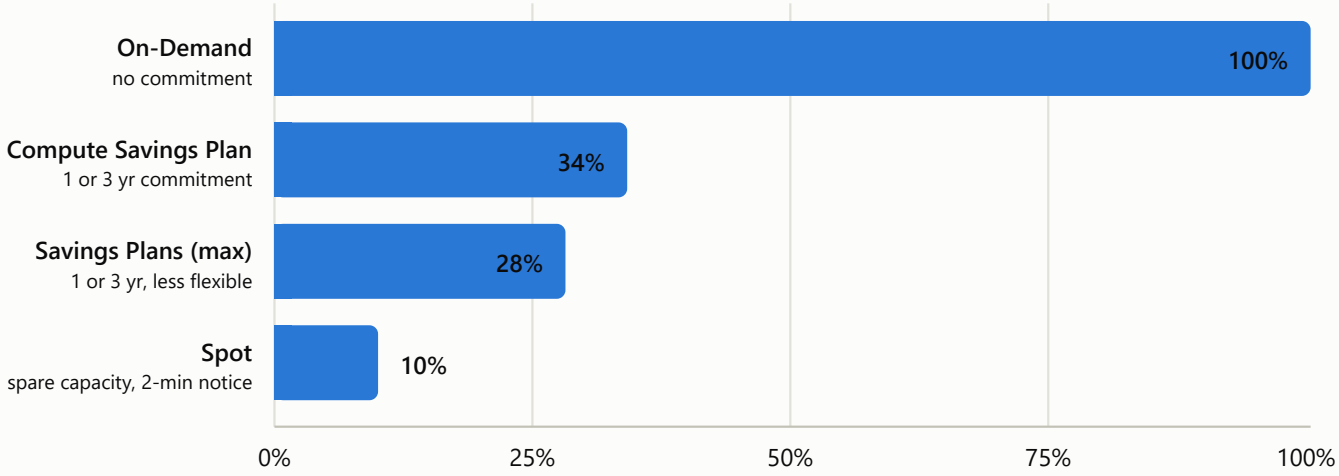
5 · State moves outward to S3 and RDS, so compute becomes disposable — and disposable compute is what Auto Scaling needs.

7 · Synchronous replication to a second AZ is what lets the database survive losing a data centre.

Nothing here changes the instance. Availability is a property of what surrounds it.

7 — PRICING

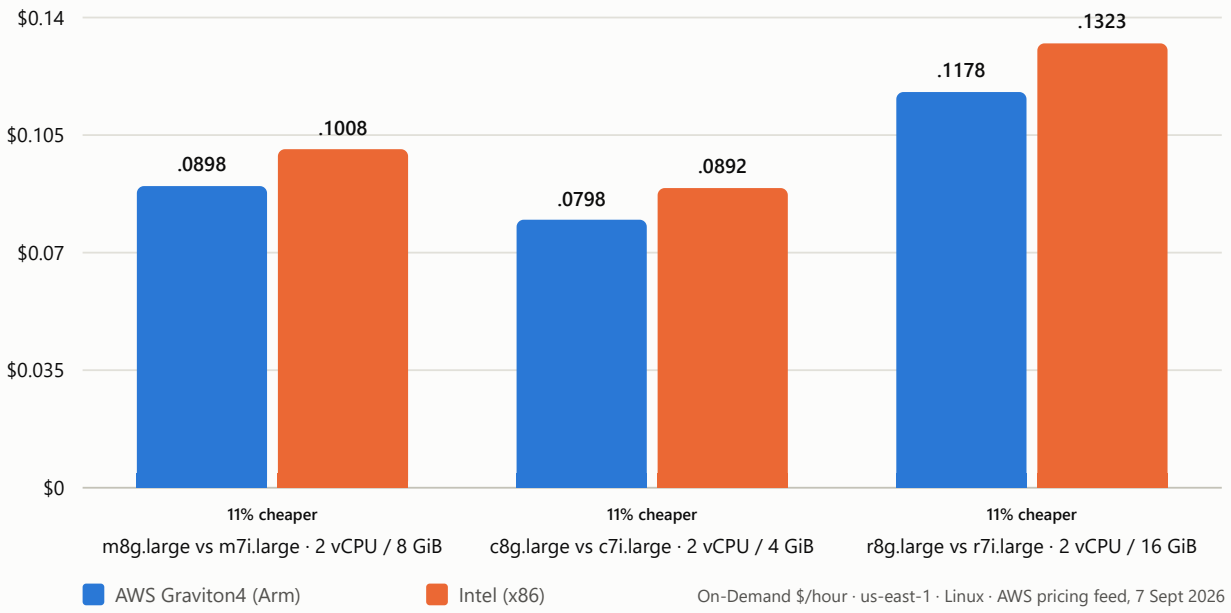
The commercial model beats every technical trick



Share of the On-Demand list price you pay. AWS's published maximum discounts — ceilings, not typical results.
Source: aws.amazon.com/ec2/pricing and aws.amazon.com/savingsplans/compute-pricing, accessed 7 Sept 2026.

Cheapest actions first: ① turn it off out of hours ② right-size it ③ commit to a Savings Plan ④ use Spot for interruptible work ⑤ *then* optimise the processor for the last 11%.

Graviton against Intel, like for like



Identical vCPU and memory either side, so this is a clean like-for-like comparison: **10.9%, 10.5%, 11.0%**. The catch is real but small — your software needs an arm64 build.

8 — USE CASES

Five workloads, and what each one should buy

USE CASE	FAMILY	PURCHASE MODEL	WHY CLOUD WINS
Web / application hosting	`m8g`, `c8g`, `t4g`	Savings Plan for baseline, On-Demand for peak	Peak-to-trough ratio — owned hardware sized for Black Friday is idle 364 days
Batch & scientific computing	`c8g`	**Spot** — up to 90% off, with checkpointing	500 instances for 1 hour costs the same as 1 for 500 hours, and finishes today
ML training & inference	`p5` train, `g6` serve	Spot or reservations for training	The alternative is a $40,179/month machine you must keep busy
Dev / test / CI	`t4g` burstable	On-Demand + scheduled shutdown; Spot for runners	Environment identical to production, costing nothing overnight
Migration & disaster recovery	Match source, then right-size	On-Demand during migration, then commit	A DR site costing a few dollars a month while idle

Report §8 · p5.48xlarge = $55.04/hr = $40,179/month if left running

9 — SECURITY

Shared responsibility, and the four mistakes

The division

AWS secures **the cloud** — hardware, hypervisor, facilities. You secure what is **in** it: guest OS patching, firewall rules, identity, encryption, application code.

Layers worth having

Identity — IAM roles, never static keys; MFA on root, then put root away.

Network — security groups *and* route tables. In slide 9 the database is unreachable by *routing*, so a bad firewall edit does not expose it.

Data — encrypt EBS, TLS at the load balancer, and restore a backup on purpose at least once.

Detection — CloudTrail names who launched it; a billing alarm is the highest-value alarm in a student account.

The four that cost marks

1. SSH open to 0.0.0.0/0. Scanners find a new public SSH port in minutes. Scope to a `/32`.

2. Mishandled private key. Downloaded once, `chmod 400`, never committed. No recovery.

3. IMDSv1 left on. One SSRF bug reads live IAM credentials from `169.254.169.254`. `HttpTokens=required` closes it.

4. Stopped, not terminated. Stopped storage still bills.

All four are one line each in `terraform/main.tf` — which is the practical argument for writing infrastructure down.

10 — COMPARISON

Same ideas, three vocabularies

CONCEPT	AWS	MICROSOFT AZURE	GOOGLE CLOUD
Virtual machine service	EC2	Virtual Machines	Compute Engine
Machine image	AMI	Managed Image / Gallery	Image
Private network	VPC	Virtual Network (VNet)	VPC network
Instance firewall	Security group	Network security group	VPC firewall rule
Failure domain in a region	Availability Zone	Availability Zone	Zone
Object storage / managed SQL	S3 / RDS	Blob Storage / Azure SQL	Cloud Storage / Cloud SQL
Committed discount / interruptible	Savings Plans / Spot	Reserved Instances / Spot VMs	Committed use / Spot VMs
In-house Arm processor	Graviton	Cobalt	Axion
Q2 2026 market share	**28%**	**20%**	**15%**

They differ in *emphasis*, not capability. **AWS**: broadest catalogue, deepest ecosystem. **Azure**: existing enterprise agreements, Active Directory, Windows licensing. **Google**: data and ML — and the fastest share gain, +3 points in six quarters.

11 — LIMITATIONS

What choosing the cloud costs you

Lock-in is not about VMs

A Linux instance is portable. The lock-in accumulates in IAM, S3 semantics, Lambda, DynamoDB — everything that makes AWS worth using. A deliberate feature of the business model.

Concentration risk is systemic

Three providers hold 67% of public cloud. A regional outage takes a visible fraction of the internet with it, and no single customer's architecture changes that.

Egress pricing shapes design

Data in is free, data out is billed. Data gravity has a direction: move compute to the data, not data to the compute.

Sovereignty precedes architecture

Region choice decides legal jurisdiction over the data. For regulated workloads that is a legal question asked before the technical one.

Cost is elastic both ways

The self-service that gives you a server in 90 seconds gives you a $40,000/month GPU cluster in 90 seconds.

Capacity is finite

“Infinite capacity” is a simplification. `InsufficientInstanceCapacity` is a real error, especially for GPUs.

12 — CONCLUSION

Provisioning is easy. That is the problem.

Launching an EC2 instance takes about three minutes on the slowest of the three paths.
Because it is that easy, the difficulty moves elsewhere.

The commercial model moves cost by up to 72%. The processor choice moves it by 11%. Make the cheap decisions first.

Availability is a property of what surrounds the instance, not of the instance itself.

The difference between the three provisioning paths is not speed — it is reviewability and reversibility.

The version worth learning

Not “launch an EC2 instance”, but:

- write the instance down as **code**
- scope its access to **what it needs**
- know what **state** it is in and what that state **bills**
- **destroy it** when the demonstration is over

Meanwhile the market underneath is growing **43% a year** and has doubled in eleven quarters. These same skills — provisioning, isolation, identity, cost discipline — sit underneath all of it.

SOURCES

Where every number came from

Market data

Synergy Research Group quarterly cloud infrastructure press releases, May 2025 – July 2026 — read directly rather than via secondary reporting. Six quarters, six releases, cited individually in the report.

Prices

The AWS On-Demand pricing feed that the public EC2 pricing page itself reads (us-east-1, Linux), retrieved 7 September 2026. Discount ceilings from aws.amazon.com/ec2/pricing and the Savings Plans page.

Definitions

NIST SP 800-145; AWS EC2 User Guide; Terraform AWS provider documentation.

The project folder

`report.html` — the full report, print to PDF

`slides.html` — this deck

`data/figures.json` — every number, with source URL and retrieval date

`terraform/` — the VM as code

`scripts/` — the CLI path and the bootstrap script

One derived value in the whole deck: Q3 2025 market size ($106.5bn), computed from Synergy's stated Q4 figure minus the stated sequential increase. It is labelled derived everywhere it appears.